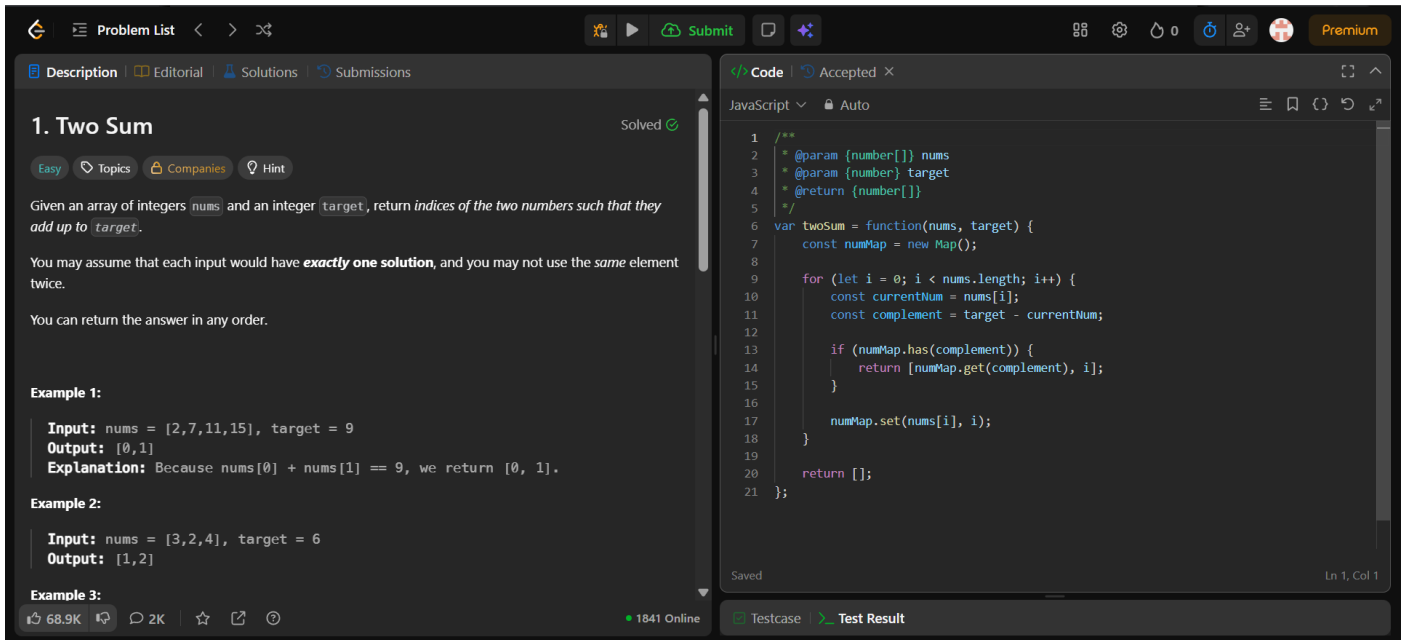


DSA - I

Question 1:- <https://leetcode.com/problems/two-sum/>

Solution: - <https://leetcode.com/problems/two-sum/submissions/2007287771/>

Screenshot:-



The screenshot displays the LeetCode website interface. On the left, the problem '1. Two Sum' is listed as 'Easy'. The description states: 'Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume that each input would have **exactly one solution**, and you may not use the *same* element twice. You can return the answer in any order.'

Example 1:
Input: `nums = [2,7,11,15]`, `target = 9`
Output: `[0,1]`
Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

Example 2:
Input: `nums = [3,2,4]`, `target = 6`
Output: `[1,2]`

Example 3:
Input: `nums = [3,3]`, `target = 6`
Output: `[0,1]`

On the right, the 'Code' editor shows a JavaScript solution using a hash map:

```
1 /**
2  * @param {number[]} nums
3  * @param {number} target
4  * @return {number[]}
5  */
6 var twoSum = function(nums, target) {
7     const numMap = new Map();
8
9     for (let i = 0; i < nums.length; i++) {
10         const currentNum = nums[i];
11         const complement = target - currentNum;
12
13         if (numMap.has(complement)) {
14             return [numMap.get(complement), i];
15         }
16
17         numMap.set(nums[i], i);
18     }
19
20     return [];
21 };
```

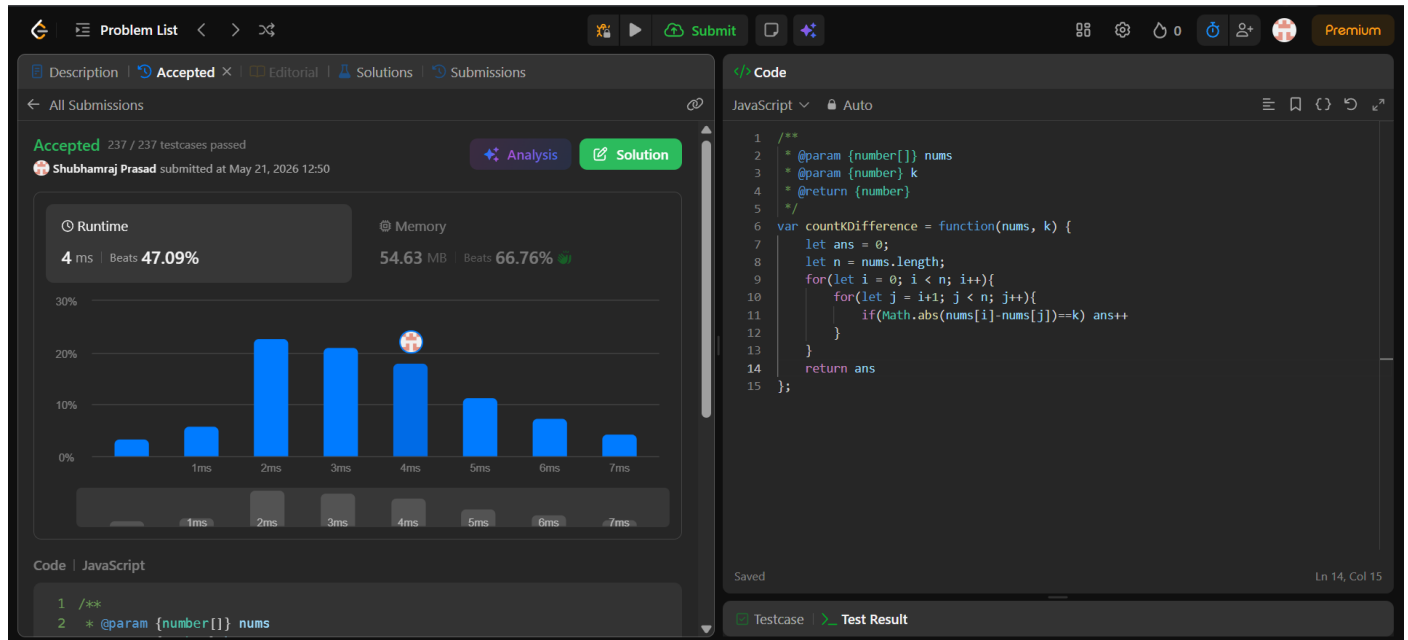
Description: -

- **Time Complexity:** $O(n)$ — We iterate through the array of length n exactly once.
- **Space Complexity:** $O(n)$ — In the worst-case scenario, we might store up to n elements in the hash map.

Question 2:- <https://leetcode.com/problems/count-number-of-pairs-with-absolute-difference-k/>

Solution: - <https://leetcode.com/problems/count-number-of-pairs-with-absolute-difference-k/submissions/2008814422/>

Screenshot:-



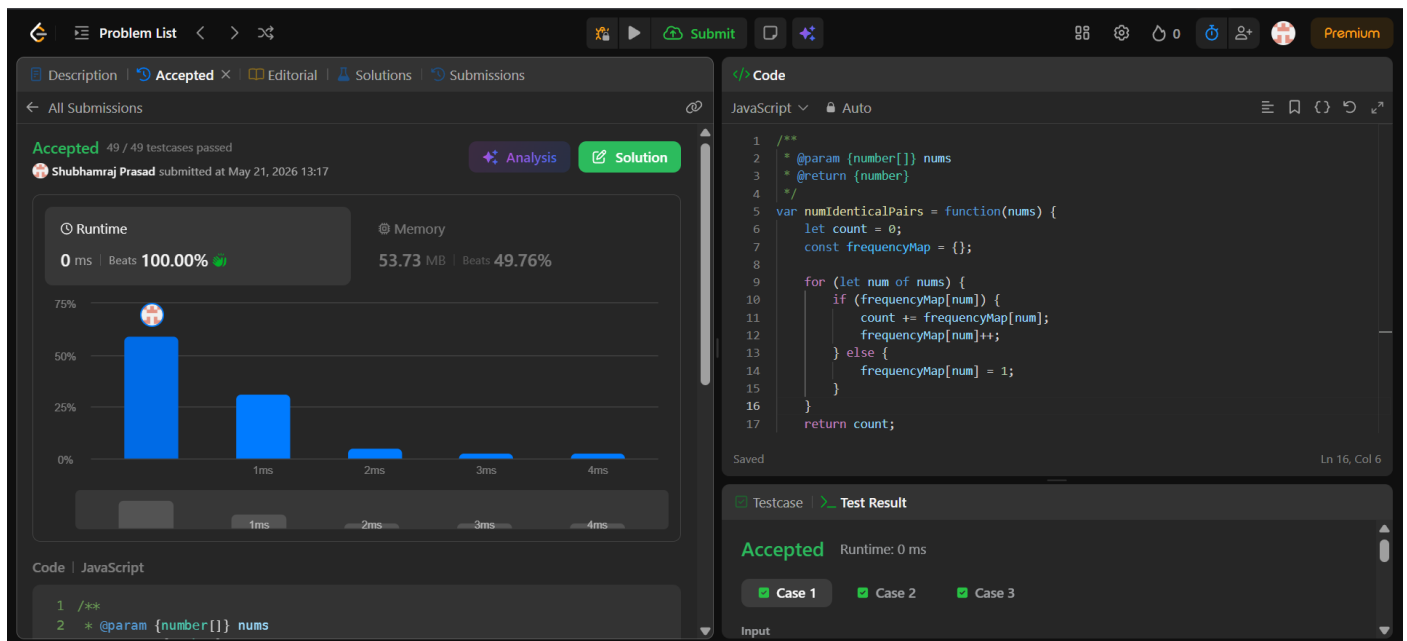
Description: -

- **Time Complexity:** $O(N^2)$ because of the nested loops. If the array has 1,000 elements, it will execute the inner condition roughly 500,000 times.
- **Space Complexity:** $O(1)$ constant space, because it only creates a couple of simple variables (ans, n, i, j) regardless of how large the array grows.

Question 3:- <https://leetcode.com/problems/number-of-good-pairs/>

Solution: - <https://leetcode.com/problems/number-of-good-pairs/submissions/2008828996/>

Screenshot:-



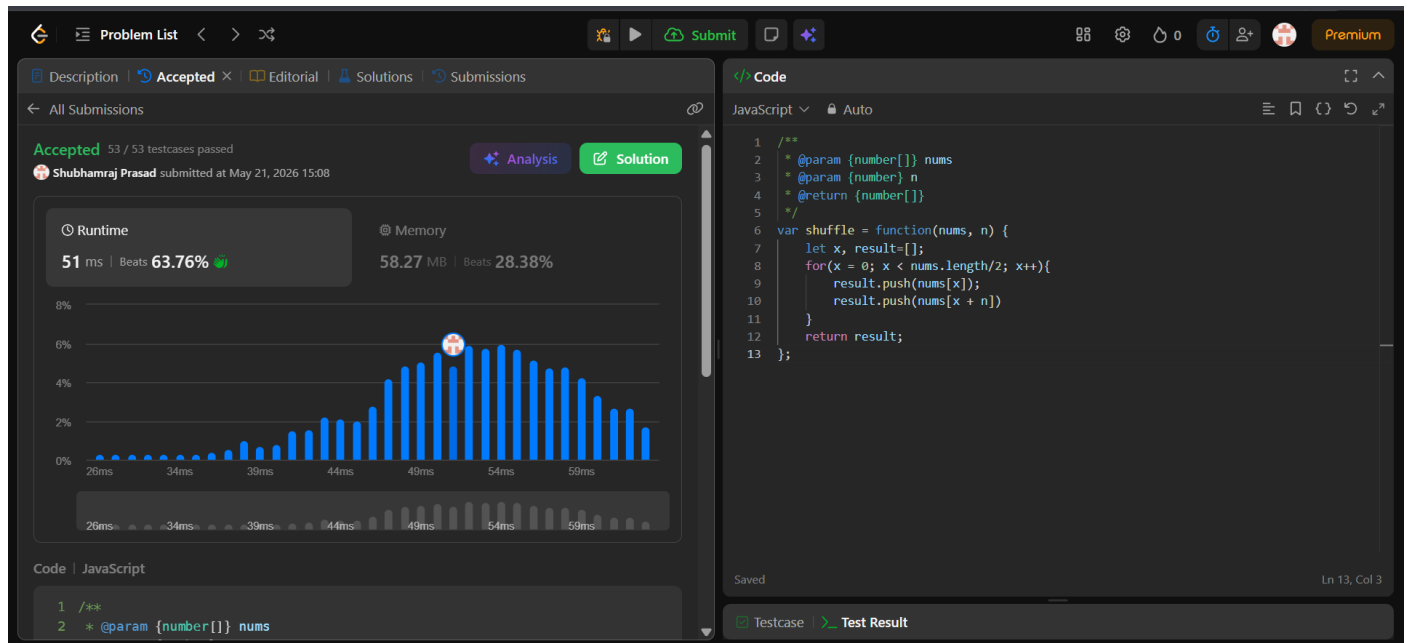
Description: -

- **Time Complexity: $O(n)$** The time complexity is linear, where n is the total number of elements in the `nums` array.
 1. **Single Pass:** The code uses a single `for...of` loop that visits each number in the array exactly once.
 2. **Instant Lookups:** Inside the loop, checking if a key exists in an object (if `frequencyMap[num]`) and updating it takes $O(1)$ constant time on average.
- **Space Complexity: $O(n)$** The space complexity is also linear, where n is the number of elements in the array.
 1. **Memory Allocation:** The code creates a new object (`frequencyMap`) to keep track of the counts.
 2. **Worst-Case Scenario:** If every single number in the input array is unique (for example, `[1, 2, 3, 4, 5]`), the map will grow to store an entry for every single element. Therefore, it will hold n distinct key-value pairs, using $O(n)$ auxiliary memory.

Question 4:- <https://leetcode.com/problems/shuffle-the-array/description/>

Solution: - <https://leetcode.com/problems/shuffle-the-array/submissions/2008903506/>

Screenshot:-



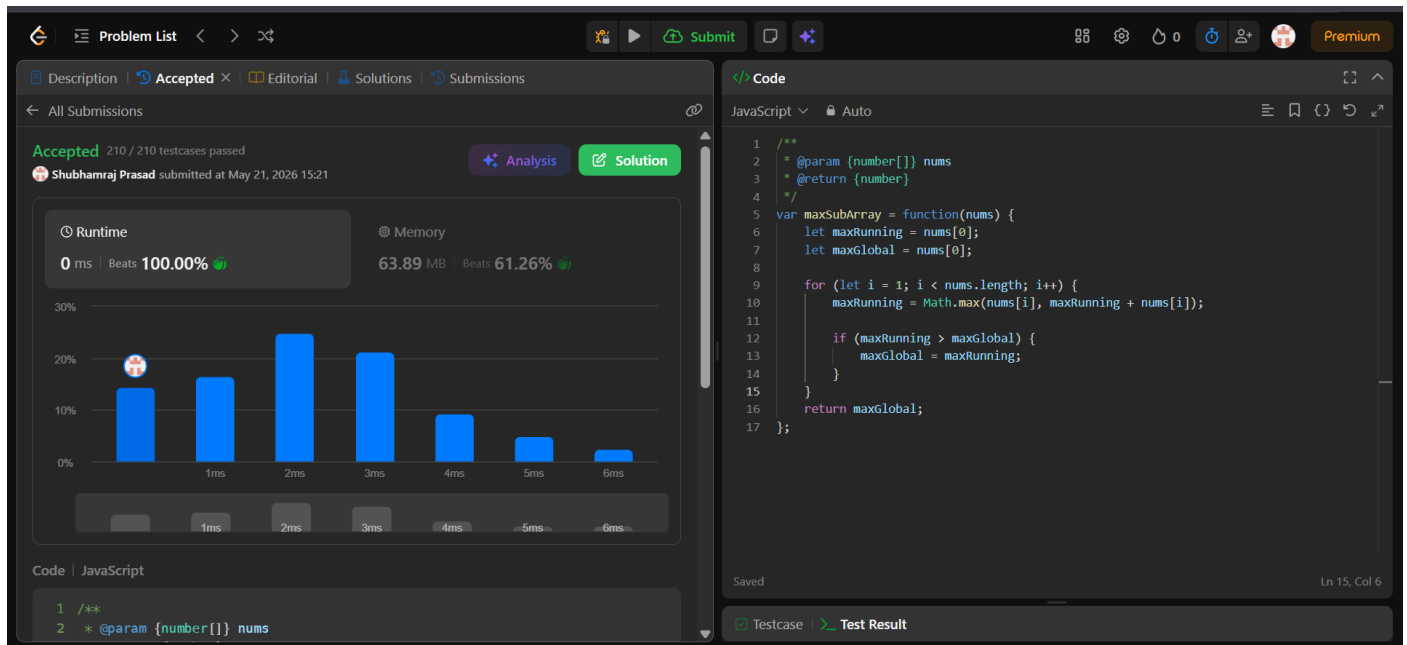
Description: -

- **Time Complexity:** $O(n)$ — The loop runs exactly n times, making it highly efficient.
- **Space Complexity:** $O(n)$ — It creates a brand new array (result) that scales linearly with the size of the input.

Question 5:- <https://leetcode.com/problems/maximum-subarray/description/>

Solution: - <https://leetcode.com/problems/maximum-subarray/submissions/2008913138/>

Screenshot:-



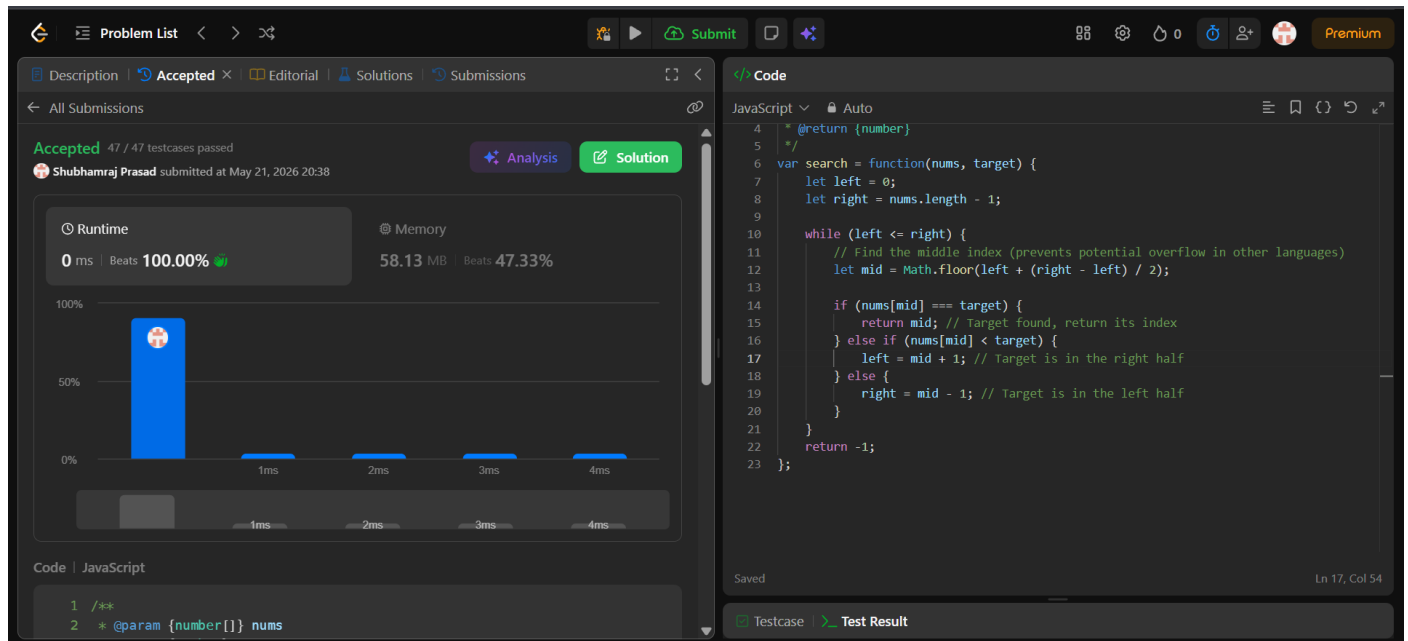
Description: -

- **Time Complexity:** $O(n)$ - The algorithm uses a single for loop to iterate through the array exactly once. Inside the loop, it only performs constant-time $O(1)$ operations (addition and `Math.max`). Thus, execution time scales linearly with the input size.
- **Space Complexity:** $O(1)$ - It only uses two variables (`maxRunning` and `maxGlobal`) to track the sums in place. No extra data structures or memory scale with the input size.

Question 6:- <https://leetcode.com/problems/binary-search/description/>

Solution: - <https://leetcode.com/problems/binary-search/submissions/2009137879/>

Screenshot:-



Description: -

- **Time Complexity:** $O(\log n)$ because the search space is cut in half with every single iteration.
- **Space Complexity:** $O(1)$ as it only uses a few pointers and modifies nothing in memory.